The Architecture of the CQRS Pattern

Different Command and Query Models

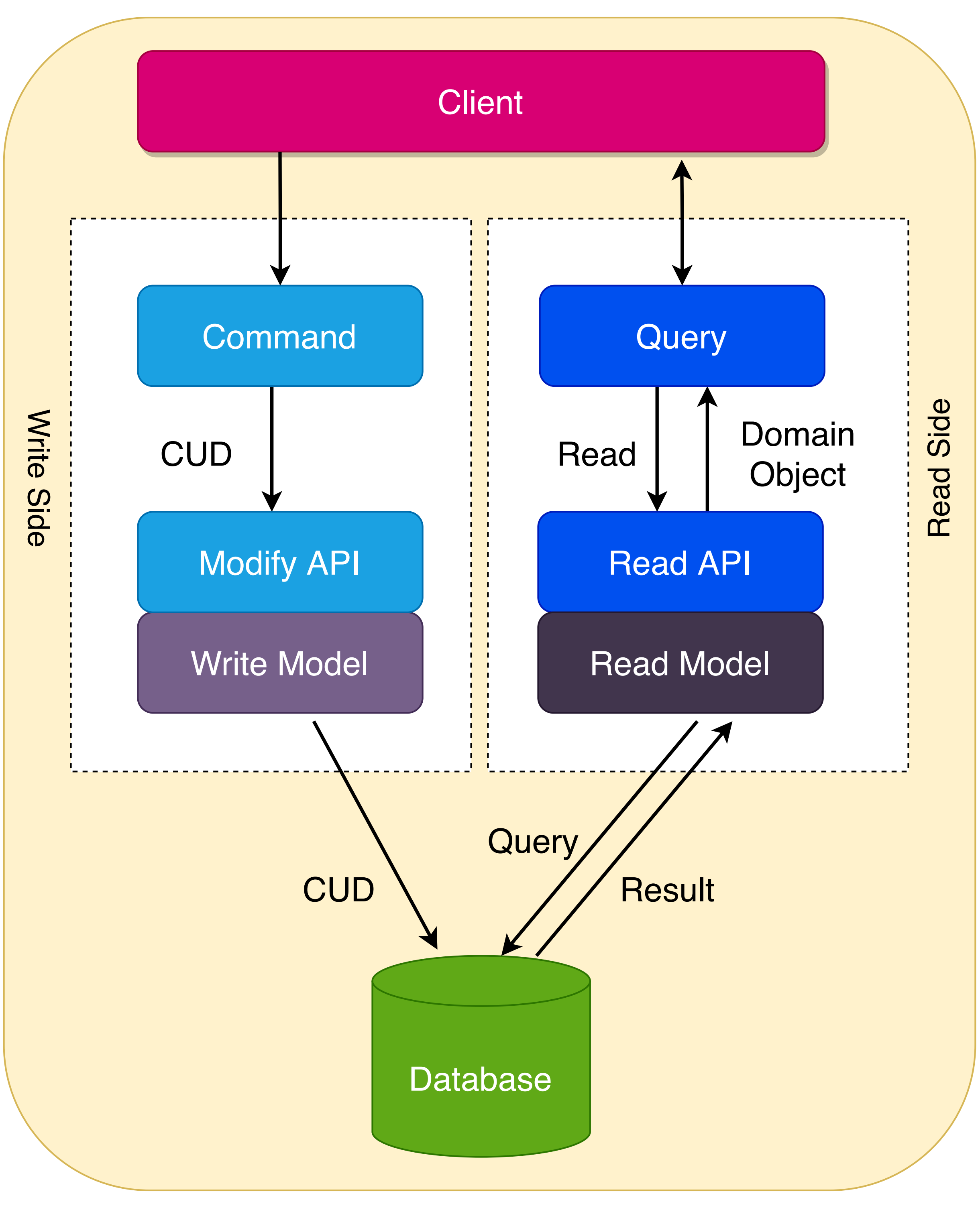
Command Model

The command model's job is to handle all write operations. It is responsible for maintaining the integrity and consistency of our data. You can think of the command model as a disciplined librarian, ensuring every book (data) is in its proper place. It receives commands, performs necessary validations, applies business rules, and alters the state of the system.

Query Model

On the other hand, the query model is responsible for handling all read operations. It provides the current state of the system to the users. The query model is like a friendly librarian, always ready to fetch the book (data) you need. It is optimized for read operations, ensuring users get the data they require efficiently and quickly.

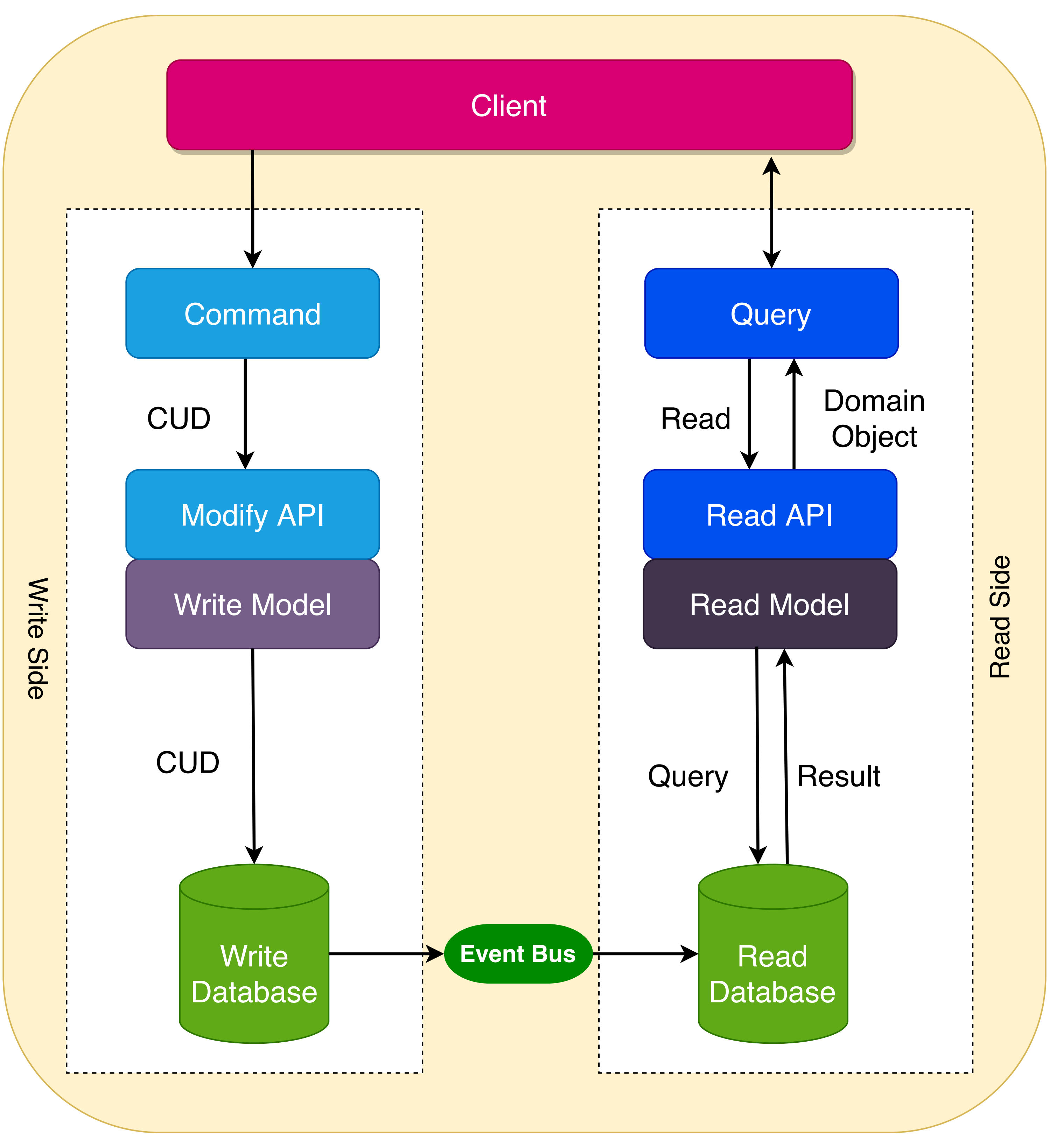
Here is what CQRS looks like with a separate Read/Write model using a single database.



Separate Command and Query Model

Different Read and Write Database

We are currently writing to a single database. To enhance flexibility and scalability, it's possible to use separate databases for read and write operations. However, this approach significantly increases complexity, as it requires careful consideration of how to propagate changes from the write database to the read database.



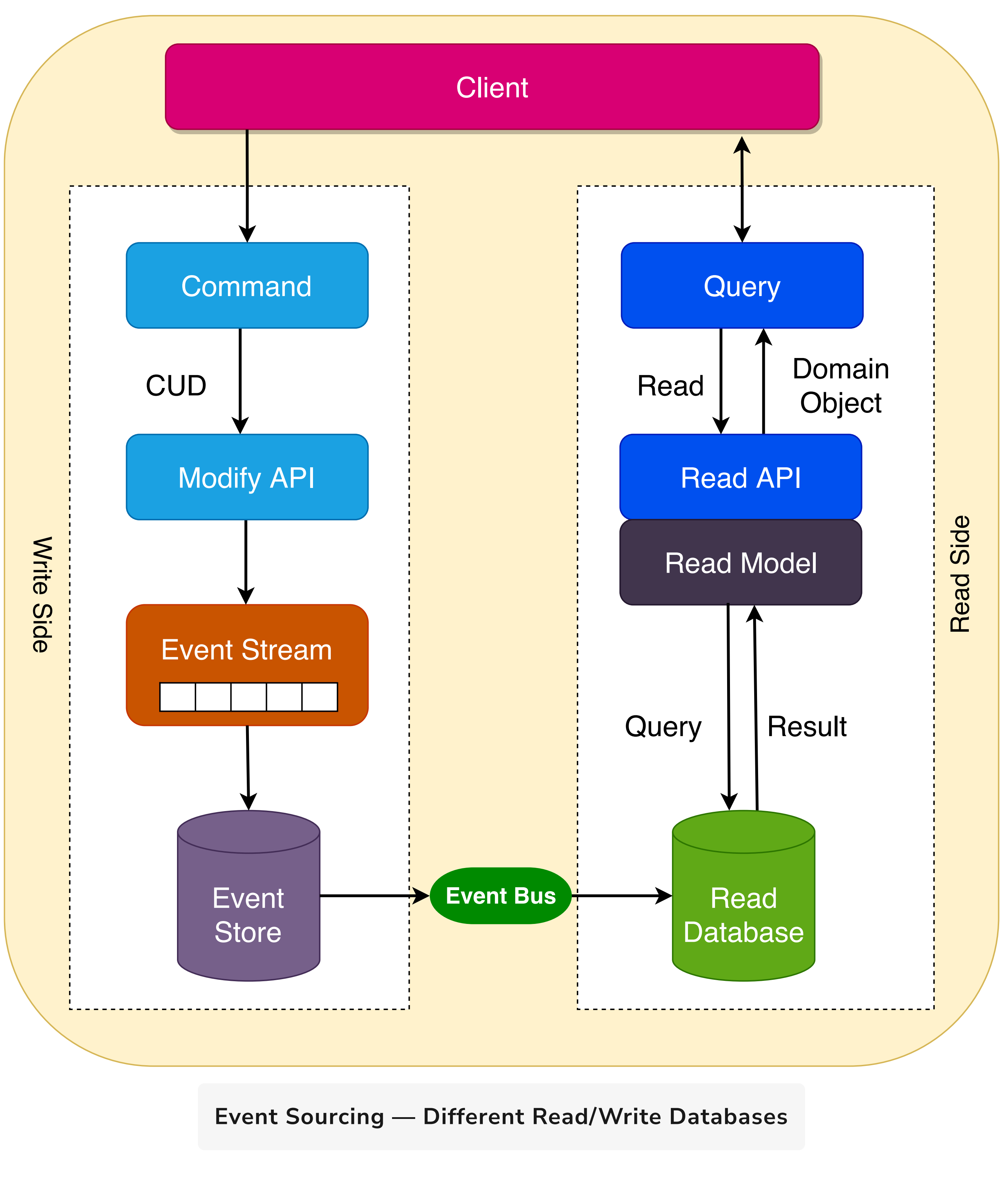
Separating read and write databases

Separate databases for Reads and Writes comes with its challenges. The lack of atomic transactions across the Read and Write models means relying on asynchronous messaging/events for eventual consistency. This raises several issues, like out-of-order event propagation, event loss, sync discrepancies, Read Model update failures, and the timing of UI updates post-writing. Implementing and managing this CQRS style demands a high degree of complexity and a deep understanding of distributed transactions. It's a choice suitable only if the app's non-functional requirements demand such an intricate solution.

Event Sourcing

Using separate databases for reading and writing presents challenges, especially in maintaining sync with Eventual Consistency. The sequence of events from the Write to Read side is crucial. For instance, if updates on a Write Model occur quickly one after another, receiving events out of order can lead to the Read model having outdated information.

Many asynchronous Message Buses prioritize availability and performance, often not ensuring the order of message delivery. Event Sourcing addresses this by adopting a different method for storing Write Models, using append-only event stores to record every action on a model. When updating, the model is refreshed by replaying its entire event history.



Event Sourcing — Different Read/Write Databases

In this approach, each Write model instance exists as an independent event stream, allowing for various data views by replaying these events. If synchronization issues arise on the Read side, rebuilding models is possible by retrieving all events from the Write side.

Event Sourcing also simplifies auditing since event streams document every change to each model. Additionally, it facilitates creating new Read Models by replaying past events. However, implementing Event Sourcing adds complexity and is a significant undertaking, especially for those new to CQRS, Event Sourcing, or distributed systems.

← Previous
CQRS Pattern: A Solution

Next →
The Inner Workings Of The CQRS Pattern

☐ Mark as Completed